Ho capito ed approvo la proposta. Ora, formalizziamo il nuovo stato di avanzamento del progetto con uno o più documenti appropriati.

Siamo partiti da una CR relativa ad un prototipo di progetto in corso di sviluppo, ora possiamo considerare quanto discusso fin'ora come la specifica di un nuovo progetto. Come vuoi procedere ?

Procederei trasformando la CR in una **baseline di specifica** del nuovo `docker-dev`, mantenendo la CR e le revisioni come documenti storici. Non continuerei ad aggiungere decisioni alla CR: ormai stiamo definendo architettura, configurazione e comportamento di un prodotto, non soltanto una modifica al prototipo.

Preparerei tre documenti principali e un registro delle decisioni.

1. Specifica funzionale

Documento: `docker-dev-functional-specification.md`

Descrive che cosa deve fare il nuovo tool, senza legarsi eccessivamente all'implementazione.

Contenuti

- obiettivi e perimetro della prima versione;
- concetti: progetto, linguaggio, servizio, runtime, container e risorsa persistente;
- operazioni disponibili;
- configurazione a due livelli;
- esecuzione con e senza servizi aggiuntivi;
- modalità effimera e persistente;
- riutilizzo del container e compatibilità di creazione;
- gestione di errori, conferme e operazioni distruttive;
- requisiti e criteri di accettazione.

Ogni requisito avrebbe un identificatore, per esempio:

```
FR-RUNTIME-001
Il tool deve comporre il runtime del linguaggio e quello del progetto.

FR-CONTAINER-004
```

```
Un container persistente fermo, appartenente al progetto e
compatibile con la configurazione richiesta, deve poter essere riavviato.
```

Questo permette di collegare requisiti, decisioni e test.

2. Specifica tecnica e dei formati

Documento: `docker-dev-technical-specification.md`

Descrive come organizzare la soluzione e, soprattutto, definisce i contratti che oggi sono ancora parzialmente illustrativi.

Contenuti

- struttura delle directory;
- orchestratore e componenti Perl;
- pipeline:

  ```
  10.setup-volumes.sh
  20.build.sh
  30.services.sh
  40.run.sh
  ```

- schema dei due `runtime.yaml`;
- schema dei manifest dei servizi;
- alias dichiarativi;
- contesto risolto e piano di esecuzione;
- rendering Template::Toolkit e aggregazione YAML;
- invocazione di Compose;
- lifecycle dei container;
- label applicative;
- normalizzazione e algoritmo del fingerprint;
- utilizzo di `inspect`;
- stato applicato, locking e gestione dei dati sensibili;
- contratti degli script e codici di uscita.

Distinguerei chiaramente:

```
Formato di configurazione dell'utente
              ≠
modello interno risolto
              ≠
documento Compose generato
              ≠
stato registrato dopo l'esecuzione
```

È una separazione importante per evitare che un unico file diventi responsabile di tutto.

3. Piano di implementazione e verifica

Documento: `docker-dev-implementation-and-test-plan.md`

Trasforma la specifica in attività realizzabili e verificabili.

Milestone proposte

Milestone	Risultato
M1 — Configurazione	Lettura, validazione e composizione dei runtime
M2 — Container singolo	Generazione Compose ed esecuzione Python senza servizi
M3 — Servizi	Catalogo, alias, template e preparazione delle dipendenze
M4 — Persistenza	Container persistenti, label, fingerprint e riutilizzo
M5 — Operazioni amministrative	Config, dry-run, down, ricreazione e cleanup
M6 — Collaudo	Test end-to-end e documentazione operativa

Il piano comprende test unitari, integrazione e scenari di errore, collegati agli identificatori dei requisiti.

La prima verifica end-to-end sarebbe il caso più semplice:

```
progetto Python
senza servizi
runtime standard
sessione interattiva
```

Solo dopo introdurrei servizi aggiuntivi e riutilizzo persistente, così da isolare i problemi.

4. Registro delle decisioni

Documento: `docker-dev-decision-register.md`

Non tutte le proposte discusse sono state approvate con lo stesso grado di dettaglio. Il registro distinguerebbe:

Stato	Significato
Approvata	Decisione esplicitamente confermata
Da definire	Direzione approvata, dettagli contrattuali mancanti
Aperta	Alternative ancora da scegliere
Superata	Proposta sostituita dalle discussioni successive

Decisioni già consolidate

- Compose come unico backend di esecuzione del progetto.
- Perl, `YAML::XS` e Template::Toolkit per i componenti comuni.
- Pipeline con `30.services.sh` e `40.run.sh`.
- `apply-templates.sh` dedicato alla consegna dei file e alla composizione della `.bashrc`.
- Runtime standard in `langs/<tipo>/runtime.yaml`.
- Runtime aggiuntivo in `<PROJECT_DIR>/.docker-dev/runtime.yaml`.
- Schema runtime semplificato, senza un sistema generale di lifecycle e hook.
- Preparazione delle risorse negli script dedicati.
- Nessuna pubblicazione implicita della porta 8000.
- Alias dichiarativi.
- Aggregazione YAML strutturata.
- Possibilità di container effimeri e persistenti.
- Riutilizzo basato sulla compatibilità di creazione.
- Label generate dal tool, fingerprint deterministico e lettura mediante `inspect`.

Dettagli ancora da formalizzare

Per esempio:

- identità stabile del progetto;
- default effimero o persistente;
- sintassi CLI definitiva;
- scelta fra `attach` ed `exec` nelle diverse operazioni;
- nome dei task one-off quando esiste il container persistente;
- versione minima di Compose e readiness;
- precedenza delle personalizzazioni;
- schema definitivo dei manifest;
- conservazione dello stato e protezione delle credenziali.

Non li trasformerei in requisiti approvati per deduzione.

5. Trattamento del prototipo esistente

Gli script allegati diventano una baseline di riferimento, non una specifica normativa del nuovo progetto.

Servono a identificare:

- comportamenti da preservare;

- parti riutilizzabili;
- difetti preesistenti;
- modifiche intenzionali;
- test di equivalenza del runtime Python.

La CR originale e la revisione 2 restano nello storico, con una nota:

```
Documento superato dalla baseline di specifica del nuovo docker-dev.
Conservato per tracciabilità delle motivazioni e delle decisioni.
```

Non eliminerei o riscriverei retroattivamente lo storico.

Come procederei adesso

Preparerei una prima baseline documentale, versione `0.1`, composta dai quattro Markdown, con:

- stato «bozza consolidata»;
- decisioni approvate riportate come tali;
- punti aperti evidenziati con identificatori;
- esempi marcati come illustrativi quando lo schema non è ancora definitivo;
- collegamenti tra requisiti, decisioni e test.

Poi risolveremmo soltanto le scelte che bloccano la prima milestone, senza pretendere di chiudere ogni dettaglio prima di iniziare.

La proposta è quindi: prima consolidare specifica funzionale, specifica tecnica, piano di implementazione/test e registro decisioni; poi implementare per milestone. Non genererei ancora codice né una nuova CR.